Practica 1:

Compilación separada

Metodología de la Programación

1º Ingeniería Informática

Grupo: E

Curso 2025/2026

Índice

1. Objetivos **2**

2. Creación de un proyecto informático **2**
2.1. Motivación 2
2.2. El entorno de programación 3
2.2.1. Netbeans 5
2.2.2. CLion 7
2.2.3. VSCode 7
2.2.4. Compilación separada con Makefile 10
2.3. Depuración 10

3. Separación de clases y compilación del programa **11**
3.1. Generación del proyecto con ficheros separados 12
3.2. Proceso de compilación 13
3.3. Estructura de los ficheros .h y .cpp 14

4. Documentando un programa con Doxygen **16**
4.1. Introducción 16
4.2. Directivas 17

5. Ejemplo **19**
5.1. La clase Par 19
5.2. La clase VectorPares 22
5.3. Fichero main.cpp 23

6. Entrega **24**

1. Objetivos

Hasta ahora, hemos tratado con programas muy pequeños que implementaban algoritmos muy concretos. Sin embargo, la realidad es que los programas que normalmente deben construirse en la vida real son muy complejos y requieren de una gran cantidad de líneas de código. Por ejemplo, una aplicación de IOS que implemente un videojuego puede tener una longitud total de 40000 líneas de código. Si toda la información se almacena en un único fichero, el fichero pasa a ser inmanejable por los programadores. Además, es mucho más cómodo repartir el código y las tareas en varios ficheros facilitando de esta forma que los programadores puedan trabajar de forma paralela. También esto hace más fácil localizar el código concreto que da error. El objetivo principal de esta práctica consiste en mostrar como podemos, mediante compilación separada, construir códigos complejos pero, a la vez, fáciles de entender y modificar. Para ello, utilizaremos ficheros diferentes para implementar las diferentes clases que formen parte de nuestro programa. Cada clase se implementará utilizando 2 ficheros distintos, un fichero .h que sirve principalmente para conocer la lista de funciones disponibles en una clase y luego un fichero .cpp donde dichas funciones se implementan. Finalmente, aparte de las parejas de ficheros que representan a las clases, habrá un fichero main.cpp que contendrá el código que crea objetos de las clases para llevar a cabo el objetivo del programa.

Cuando trabajamos con programas de gran envergadura, es importante que el código esté correctamente documentado. El objetivo de esto es doble. Por un lado, cuando documentamos de forma correcta un programa que hemos creado, podemos volver al código y reutilizarlo más adelante sin problemas. Cuando dicho código no está documentado, nos es mucho más difícil recordar lo que hicimos y por qué lo hicimos. Por tanto, el proceso de reutilización del código nos llevará mucho más tiempo y tendremos mucha más propensión a cometer errores. Por el contrario, si el código está bien documentado, nos será fácil comprenderlo. Por otro lado, cuando trabajamos en códigos de gran envergadura, es normal que haya dos o más programadores implicados. Si estos programadores comentan adecuadamente su código, entonces nos será mucho más fácil entender el código que ha realizado alguien ajeno a nosotros. Sino, es muy posible que haya partes de código que no consigamos entender por no comprender su utilidad. En esta práctica estudiaremos como documentar el código utilizando la herramienta Doxygen. Doxygen es una herramienta que nos permite añadir comentarios sobre el código y nos genera documentación útil sobre el programa que estamos diseñando. El uso de Doxygen sólo conlleva el uso de unas directivas muy simples durante el proceso de comentado del código. El uso de estas directivas ayuda a establecer un proceso de documentación cómodo y estructurado.

2. Creación de un proyecto informático

2.1. Motivación

En la asignatura de FP hemos aprendido a cómo crear un fichero de código en C++ y compilarlo. Sin embargo, como ya hemos comentado, esta no es la manera en la que se suelen llevar a cabo los proyectos de programación de mediana o gran envergadura. Aunque es posible llevar a cabo el proceso de compila-

ción separada que veremos en la sección 3 únicamente mediante la creación de los ficheros y llamadas al compilador, es mucho más común y sencillo el uso de un entorno de programación. Los entornos de programación, por lo general, trabajan con proyectos. Cada proyecto sirve para construir un programa concreto. Este programa que representa el proyecto, a su vez, estará formada por varios ficheros .cpp y varios .h. Una de las ventajas de trabajar con proyectos es que los entornos de programación compilan todos los ficheros asociados al proyecto de forma conjunta. Por tanto, podemos olvidarnos de cómo hemos de combinar los ficheros para generar el ejecutable final. Aun así, es importante ir generando o añadiendo los ficheros al proyecto de forma adecuada si no queremos generar errores. Cuando trabajamos con proyectos, es importante tener en cuenta los siguientes puntos:

- Por lo general, cada proyecto queda representado por un directorio que contiene toda la información del programa. Sin embargo, los entornos de programación permiten añadir ficheros externos que están localizados fuera de ese directorio. Hacer esto es una mala práctica de programación ya que, una vez entregamos en prácticas un proyecto o movemos el proyecto de sitio, éste deja de funcionar por no encontrar los ficheros que había fuera de esa carpeta. Por tanto, es importante asegurarse de que la carpeta de proyecto con la que trabajamos contiene todos los ficheros que se utilizan en él y no hay referencias a ficheros externos.

- Los proyectos se componen de multitud de ficheros. Algunos de ellos son los ficheros que configuran la correcta compilación del proyecto y, sin ellos, el proyecto no puede abrirse ni compilarse. Es importante recordar que, además, dichos ficheros no contienen toda la información del programa por lo que, para que una entrega de prácticas sea válida, debe entregarse la carpeta entera del proyecto comprimida.

- Es muy importante que, a la hora de añadir nuevos ficheros al proyecto, se utilicen rutas relativas con respecto a la carpeta del proyecto. Esto hará que la carpeta del proyecto sea portable y pueda moverse el proyecto a otro ordenador.

- Cada tipo de fichero debe de guardarse en una carpeta distinta dentro del proyecto. Esto hará que la carpeta del proyecto esté organizada.

2.2. El entorno de programación

Para la realización de las prácticas, es aconsejable el uso de un entorno de programación. En esta subsección presentaremos los tres entornos de programación más populares que nos permiten trabajar con C++ y crear proyectos con múltiples ficheros. Además, mostraremos como crear un makefile que nos permitirá trabajar sin entorno de programación, aunque esto es menos recomendado debido a que es bastante más incómodo.

Se recomienda realizar la instalación del programa en una distribución de Linux. A lo largo de la carrera utilizaréis Linux bastantes veces con lo que es bueno que vayáis instalándolo y aprendiendo a usarlo. Una de las distribuciones más usadas es Ubuntu: `https://ubuntu.com/download`. Recomendamos bajar

la versión LTS (Long Term Service) ya que no suele haber grandes diferencias entre esa versión y la más moderna y la versión LTS está más testada y es menos dada a fallos. Todos los entornos de programación que instaléis os van a pedir que el compilador g++ esté instalado. En el caso de la distribución Ubuntu, se puede instalar automáticamente usando el siguiente comando en la terminal del sistema:

```
sudo apt-get install g++
```

Sin embargo, también es posible utilizar un entorno de programación en Windows si se instala un compilador de C++ y un entorno de terminal. Una de las mejores opciones existentes pasan por la instalación de los siguientes 2 programas:

- **Compilador C++ para Windows (WinBuild)**: `http://win-builds.org/doku.php`
- **Terminal para compilar (Cygwin)**: `https://www.cygwin.com/`

La instalación de Cygwin por sí sólo también funciona ya que se puede configurar la instalación para que contenga el compilador. Una alternativa a estos programas, por si ninguna de las dos herramientas comentadas os funcionara, sería utilizar MinGW32 y MSYS. Se pueden bajar de este enlace:

`https://sourceforge.net/projects/mingw/files/`.

De todas formas, recomendamos la instalación en Linux ya que es más sencilla y es un entorno que usaréis en el futuro. En Internet pueden encontrarse multitud de guías para realizar su instalación. Aquí abajo os dejo el enlace a una de ellas:

`https://www.muylinux.com/2020/05/21/guia-instalacion-ubuntu-20-04-lts/`.

Una opción intermedia consiste en la instalación de Ubuntu sobre Virtual Box en Windows. Para ello, se puede seguir el siguiente tutorial:

`https://osl.ugr.es/2020/09/29/como-instalar-ubuntu-en-virtual-box/`.

También hay muchos videostutoriales que explican como llevar a cabo este proceso en Youtube.

A continuación, mostraremos como instalar cada uno de los diferentes entornos de programación recomendados.

2.2.1. Netbeans

Una de las posibilidades es el uso de Netbeans. Netbeans está disponible en las aulas y se puede descargar gratuitamente para trabajar en casa. Podéis descargarlo desde este enlace: `https://netbeans.apache.org/download/archive/index.html`.

IMPORTANTE: Es necesario instalar, primeramente, una versión anterior a la 21 ya que las posteriores no permiten la instalación del plugin de C++. Una vez instalado el plugin, se puede actualizar Netbeans a la última versión.

Dentro de las aulas de prácticas, podéis acceder al programa entrando con vuestro login y contraseña y eligiendo el sistema operativo Ubuntu 20.04 o 18. Se puede, y de hecho, se recomienda, que cada alumno trabaje en su propio portátil ya que eso evita tener que enviar ficheros de un ordenador a otro. En el siguiente enlace, tenéis un vídeo que os ayudará a entender cómo funciona el programa y cómo podemos generar un nuevo proyecto con él:
`https://drive.google.com/file/d/1KgAAz5xdNer0Qc64-hrdAerELHkHKFpz/view?usp=sharing`
Se recomienda poner este video a 720p para una correcta visualización.

Es importante, para la correcta instalación del plugin de C++, la instalación de OpenJDK 11 **(esa versión, no una posterior)**. Pueden verse instrucciones para su instalación en el siguiente enlace: `https://openjdk.org/install/`

Una vez instalado OpenJDK 11, es necesario instalar una versión de Netbeans anterior a la 21 ya que las posteriores a la 21 no son compatibles con OpenJDK 11 y no permiten instalar luego el plugin de C++. Una vez instalado ael plugin de C++ se puede actualizar tanto el OpenJDK como el entorno Netbeans.

Si aún así, preferís instalar el programa en Windows, el siguiente vídeo enseña cómo se deben instalar y configurar Cygwin y Netbeans para su correcto funcionamiento en Windows:
`https://drive.google.com/file/d/1Vl92vYhe2psYTTuPhyagmBEzKhQohyGV/view?usp=sharing`

Se recomienda cambiar la calidad del video a 720p para una mejor visualización. Este vídeo explica la instalación sobre Netbeans 8.2 que es el disponible en prácticas y que muy recientemente dejó de estar disponible en la Web de Netbeans. La instalación del plugin de C++ de la versión Netbeans 12.6 se realiza de forma muy similar, en el mismo menú de Plugins. El único cambio es que, dentro de plugins, hay que ir a settings y marcar **Netbeans 8.2 Plugin Portal** tal y como muestra la Figura 1. Posteriormente se debe instalar el plugin C/C++ **cuyo source es Netbeans 8.2 Plugin Portal**. Hay otro, pero está en desarrollo y no va bien.

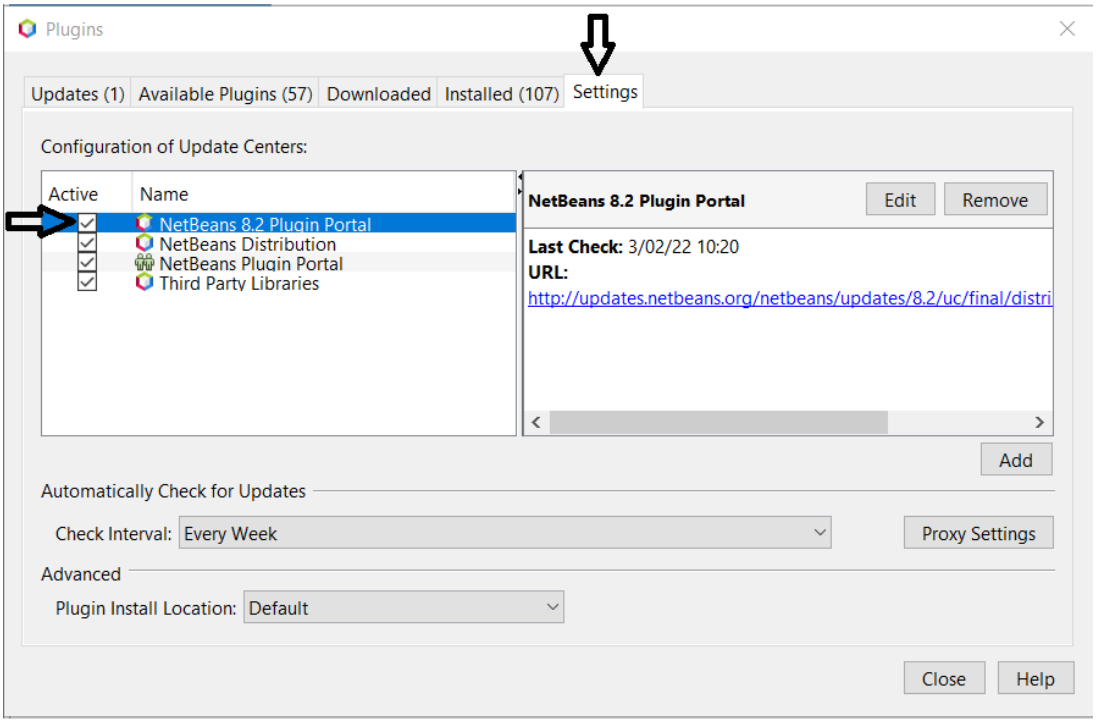

Figura 1: Instalación del plugin en Netbeans 12.6.

Dentro de los programas que indica el vídeo que se instalen dentro de Cygwin, también puede seleccionarse el paquete de Doxygen. El proceso de instalación de Cygwin para plataformas Linux no es necesario, por lo que no se incluye vídeo en ese caso. En Linux simplemente hay que bajarse el instalador de Netbeans de linux y ejecutarlo para llevar a cabo la instalación. Una vez instalado se puede añadir el plugin de C++ de la misma forma que indica el vídeo de Windows. Esto es, usando el menú Tool-> Plugins y seleccionando el plugin C/C++ en la ventana de Available Plugins.

Finalmente, os dejo un par de enlaces con una máquina virtual que ya tiene el programa instalado. Puede utilizarse si instaláis el programa virtualbox en vuestro ordenador:

- Enlace a la guía: `https://drive.google.com/file/d/1CNMMm5XoNtCsmYAjIF_rWGmqM8JDdUhK/view?usp=sharing`
- Enlace a la máquina virtual: `https://drive.google.com/file/d/1gbGjah1DGrNCRvayqGzKUqXx8iqkcOF7/view?usp=sharing`

2.2.2. CLion

CLion no está disponible en las aulas de prácticas pero es de las opciones más sencillas de instalar, configurar y utilizar así como uno de los entornos de programación más potentes para programar en C++.

Puede descargarse CLion en el siguiente enlace: `https://www.jetbrains.com/clion/`

Nótese que existe tanto la versión para Windows como la versión para Linux. Como hemos comentado antes, es preferible utilizar la versión de Linux. Para instalar CLion en Ubuntu sólo hace falta ejecutar la siguiente instrucción en una terminal:

```
sudo snap install clion –classic
```

Una vez instalado, si se instaló previamente el compilador g++ y make, el entorno encuentra automáticamente la localización del compilador y puede comenzar a usarse. Nótese que, si no se hace nada más, CLion está funcionando con una licencia de prueba. Para obtener una licencia no comercial para proprósitos educativos, es necesario registrarse en la web de jetbrains con la cuenta de correo de la ugr y solicitar la licencia. Una vez activada la licencia en el programa puede utilizarse sin ningún problema.

2.2.3. VSCode

VSCode está disponible en las aulas de prácticas y, aunque es algo más complicado de configurar que CLion, es una buena opción para programar en C++ ya que VSCode es un entorno de programación ligero y apto para cualquier lenguaje de programación. Al igual que CLion, existen versiones tanto para Windows como para Linux y, de igual forma, recomendamos su uso en linux.

VSCode puede descargarse gratuitamente desde el siguiente enlace:

`https://code.visualstudio.com/`

En Ubuntu, es posible realizar la instalación descargando el archivo .deb de la página web y ejecutando el siguiente comando:

```
sudo apt install code_vXXX.deb
```

Donde XXX es el número de la versión que hayáis descargado. Una vez instalado, ya podéis ejecutar el entorno. Sin embargo, es necesario configurarlo para que pueda compilar código en C++. Para ello,

podemos seguir los siguientes pasos:

1. En el apartado de extensiones, instalar la C/C++ extension Pack. Preferiblemente la recomendada por Microsoft.

2. Crear una nueva carpeta para el proyecto y abrirla con el VSCode usando la opción de OpenFolder.

3. Crear un nuevo fichero main.cpp en la carpeta.

4. Escribir código de Hola mundo válido y darle a ejecutar. En la primera ejecución te da varias opciones de compilación. Se recomienda utilizar la que ponga g++ con la versión más reciente.

5. El fichero .cpp se compila y ejecuta.

Esta configuración nos permite ejecutar el código de un único fichero .cpp. Sin embargo, para realizar un proceso de compilación separada como el que describe este guión, hay que realizar varios cambios en el fichero tasks.json que se genera tras la primera ejecución del programa. Dichos cambios puede verse en la Figura 2.

```
1  {
2      "tasks": [
3          {
4              "type": "cppbuild",
5              "label": "C/C++: g++-13 build active file",
6              "command": "/usr/bin/g++-13",
7              "args": [
8                  "-fdiagnostics-color=always",
9                  "-g",
10                 "${file}",
11                 "-I./include",
12                 "./src/*.cpp",
13                 "-o",
14                 "${fileDirname}/${fileBasenameNoExtension}"
15             ],
16             "options": {
17                 "cwd": "${fileDirname}"
18             },
19             "problemMatcher": [
20                 "$gcc"
```

Figura 2: Configuración del fichero tasks.json para hacer compilación separada en C++.

Además, es necesario incluir en la variable includePath del fichero c_cpp_properties.json el directo-

rio en donde incluyamos todos los ficheros .h. Puede verse la configuración final del fichero en la Figura 3. Para que ese fichero se añada a la carpeta de proyecto, se puede seleccionar uno de los errores que de algún cpp en la inclusión de un .h y seleccionar la opción de cambio.

```
{} c_cpp_properties.json ×   {} tasks.json   C/C++ Configurations
.vscode > {} c_cpp_properties.json > [ ] configurations > {} 0 > compilerPath
1  {
2      "configurations": [
3          {
4              "name": "Linux",
5              "includePath": [
6                  "${workspaceFolder}/**",
7                  "${workspaceFolder}/include/**"
8              ],
9              "defines": [],
10             "compilerPath": "/usr/bin/gcc",
11             "cStandard": "c17",
12             "cppStandard": "gnu++17",
13             "intelliSenseMode": "linux-gcc-x64"
14         }
15     ],
16     "version": 4
17 }
```

Figura 3: Configuración del fichero c_cpp_properties.json para hacer compilación separada en C++.

Hay otras maneras de resolver este problema pero la descrita aquí es la que considero más sencilla y directa. Nótese que la solución descrita consiste en un proyecto con las siguientes características:

- Se almacenan todos los ficheros .cpp menos el main.cpp en un directorio src dentro de la carpeta de proyecto.
- Los .h se almacenan en un directorio include.
- El directorio raíz contiene el main.cpp y el ejecutable generado tras la compilación.

2.2.4. Compilación separada con Makefile

Los entornos de programación son de gran ayuda a la hora de programar y nos permiten gestionar grandes proyectos de programación con multitud de ficheros de código de forma cómoda. Sin embargo, no son estrictamente necesarios a la hora de programar código en varios ficheros de manera separada. Podemos usar un editor de textos enriquecido y luego realizar la compilación por terminal utilizando un Makefile. Para ello, podemos seguir los siguientes pasos:

- Cerciorarnos de que el fichero binario make está instalado. Sino, ejecutar el siguiente comando para instalarlo:

  sudo apt-get install make

- Creamos una carpeta que será nuestra carpeta de proyecto. Dentro de esa carpeta creamos un directorio src para guardar los ficheros .cpp y un directorio include para almacenar los ficheros .h. El main.cpp se puede quedar en la carpeta raíz.

- Creamos un fichero de texto llamado Makefile (sin extensión) en el que se debe incluir información similar a la mostrada en la Figura 4.

- Finalmente, desde el directorio del proyecto, ejecutamos make.

- Se genera el ejecutable que podemos ejecutar desde el terminal.

2.3. Depuración

A continuación, veremos cómo podemos utilizar el depurador del entorno de programación. Gracias al depurador, es posible saber en cada momento de la ejecución, qué instrucción se está ejecutando y qué valores tienen las variables declaradas hasta el momento. Mostraremos como hacerlo en Netbeans pero en verdad, en el resto de entornos de programación, el funcionamiento es muy similar. En el siguiente vídeo, tenéis un pequeño tutorial corto que muestra cómo funciona el depurador de Netbeans:
`https://drive.google.com/file/d/1KBmB7zrNlMjfCE0YJB5yQOQnAi3wnXD3/view?usp=sharing`

A continuación, detallamos la terminología utilizada más importante:

- **Breakpoint**. Colocamos un breakpoint, en aquella línea en donde queremos que el programa se detenga. Es muy útil colocar breakpoints, por ejemplo, dentro de bucles for, ya que de esa manera, en cada iteración, podemos ver que valores toman cada una de las variables. Encontrar una variable que adquiere valores no esperados permite identificar donde está el problema. Colocamos un breakpoint pinchando en el número de línea del código.

- **Botón de depuración**: Una vez colocados los breakpoints, pulsamos el botón de ejecución con de-

```
Abrir  Makefile
~/pruebacom... cionmakefile

TARGET = $(EJ)main
EJ= ./dist/
BUILD= ./build/
SRC= ./src/
INC = ./include/
CFLAGS = -g -I$(INC)
OBJS =  $(BUILD)VectorPares.o \
        $(BUILD)Par.o \
        main.cpp

$(TARGET) : $(OBJS)
        mkdir -p $(EJ)
        g++ $(CFLAGS) $(OBJS) -o $(TARGET)

$(BUILD)%.o : $(SRC)%.cpp
        mkdir -p $(BUILD)
        g++ -c $(CFLAGS)  $< -o $@


.PHONY: clean
clean:
        rm -r $(BUILD) $(EJ)
```

Figura 4: Ejemplo de makefile para la compilación del ejemplo de VectorPares.

puración (Debug project). De esta manera, el código se irá ejecutando y deteniéndose cada vez que vea un breakpoint. En la ventana de abajo del entorno, podemos observar varias pestañas. La más importante es la de *Variables*, que incluye los valores de todas las variables disponibles.

- **Breakpoint condicional**: Podemos establecer propiedades especiales para que se active un breakpoint. Por ejemplo, *activar el breakpoint cuando la variable i valga 15*. Es muy útil cuando queremos comprobar qué pasa en una iteración concreta de un bucle.

3. Separación de clases y compilación del programa

En esta sección aprenderemos cómo realizar un proyecto de programación con compilación separada. Primero veremos como generar un proyecto con ficheros separados, luego cómo se lleva a cabo el proceso de compilación y terminaremos comentando la estructura que deben tener los ficheros que debemos crear.

3.1. Generación del proyecto con ficheros separados

Una vez que hemos creado el proyecto, debemos ir generando e implementando las diferentes clases que formarán parte de nuestro programa. Para ello, podemos utilizar archivos ya creados o crear una nueva clase e implementarla desde 0.

Una vez se han añadido los ficheros, tenemos 3 conjuntos de ficheros:

- **Ficheros .h**: Estos ficheros, llamados de cabecera, almacenan la estructura de la clase. Contienen los datos miembros y las cabeceras de las funciones de, generalmente, una única clase. En caso de no estar utilizando el fichero .h para implementar una clase sino que se use para definir un conjunto de funciones, este fichero contendrá una lista de cabeceras funciones.

- **Ficheros .cpp**: Implementan las funciones de la clase o del conjunto de funciones definido en un fichero .h.

- **Fichero main**: Incluye los ficheros .h de las clases o conjuntos de funciones que se vayan a utilizar e inicia la ejecución.

Aunque el entorno de programación nos hace una separación clara de éstos, la realidad es que por un lado tenemos la organización lógica que realiza el entorno de programación de los ficheros y por otro lado está la manera en que dichos ficheros se almacenan en disco duro. Aunque el entorno de programación ordene los ficheros de forma automática cuando trabajamos con ellos, es recomendable que seamos nosotros quienes configuremos una organización clara a nivel de disco duro. Esto facilitará la búsqueda manual de dichos ficheros. Lo más recomendable es crear un directorio por cada tipo de fichero. De esta forma, estarán más organizados. Un posible ejemplo de organización puede verse a continuación:

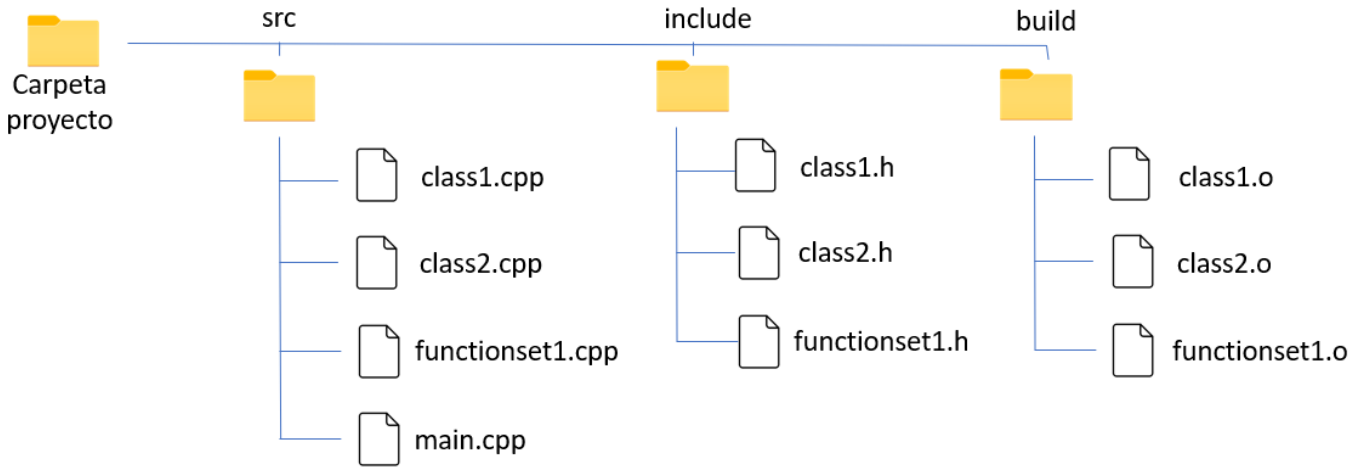

Figura 5: Estructura de carpetas en disco duro de un proyecto.

Aparte de estos directorios, es posible incluir otros tales como **data**, para ficheros de datos, **doc**, para los ficheros de documentación, o **dist** para almacenar los ficheros compilados (esto último lo hace automáticamente Netbeans).

3.2. Proceso de compilación

El proceso de compilación de los ficheros sigue el esquema mostrado en la Figura 6. Tal y como

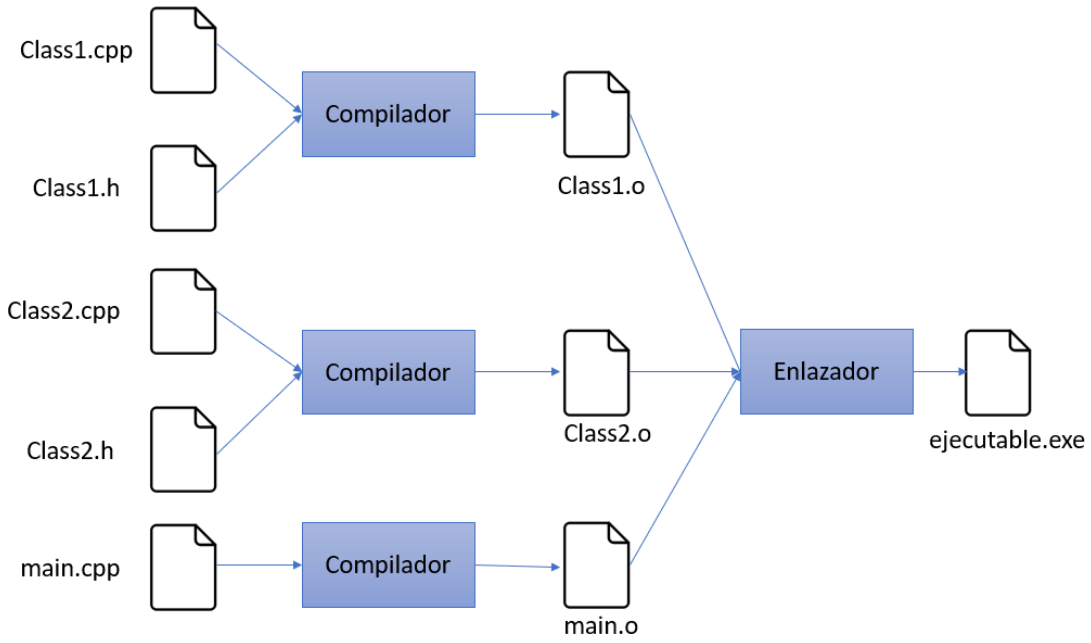

Figura 6: Esquema del proceso de compilación.

puede observarse en la imagen, cada clase se compila por separado generando un fichero de tipo .o. Los ficheros .o son objetos compilados similares a los ejecutables. La principal diferencia radica en que un fichero .o, a pesar de ser código compilado, no puede ejecutarse hasta no ser enlazado con un fichero compilado que contenga una función main. Por tanto, cada clase o conjunto de funciones se compila por separado generando ficheros .o. Una vez todos los ficheros .o están disponibles, el enlazador los combina generando el ejecutable final. Es importante tener en cuenta, a la hora de compilar un proyecto, que debe indicarse al entorno de programación que vuelva a crear todos los ficheros .o. Aunque, por lo general, el entorno de programación crea aquellos ficheros .o donde ve que los .cpp han sido modificados, puede haber casos en los que, aun modificando los ficheros .cpp o .h, el nuevo fichero .o no se genere. Por tanto, si había un error de ejecución, este puede seguir apareciendo a pesar de haber sido resuelto. Se recomienda, por tanto, indicar al entorno de programación que realice una compilación completa del proyecto antes de cada ejecución.

3.3. Estructura de los ficheros .h y .cpp

A continuación, pasaremos a ver la estructura exacta que deben seguir los ficheros .h y .cpp para que la compilación sea correcta. Como ya hemos comentado, los ficheros .h contendrán la estructura de la clase así como todos las cabeceras de los métodos disponibles. También será aquí donde indiquemos qué métodos son públicos y cuáles son privados. El fichero .cpp incluirá la implementación de todas las funciones de la clase. En el caso de tratarse de una biblioteca de funciones, simplemente incluiremos en el .h las cabeceras de las funciones. Dichas funciones serán implementadas en el fichero .cpp.

Pasemos a continuación a ver qué apartados debemos incluir en los diferentes ficheros. Comencemos por los ficheros .h. Para ello, podemos ver el fichero de ejemplo Persona.h que incluye datos personales. La clase sólo tiene los operadores de acceso más básicos, tal y como podemos ver en la Figura 7. De dicha figura, es importante tener en cuenta los siguientes puntos:

- Lo primero que debemos observar es que **el fichero debe tener el mismo nombre de la clase**. De esta manera, será sencillo identificar qué fichero contiene qué clase.

- Todo el fichero debe estar englobado por una **directiva #ifndef #define <código> #endif**. Cuando programamos con ficheros, muchas veces no sabemos si un determinado fichero .h está incluido o no en el fichero del código. Si no usamos la directiva #define, entonces puede darse el caso en el que las cabeceras de las funciones se agreguen varias veces. Gracias a la directiva #ifndef #define, aunque agreguemos el .h usando la directiva include varias veces, las cabeceras sólo se agregan una vez. Esto es porque, al agregar por primera vez el fichero .h se crea un entorno con el nombre indicado y, al volver a agregarlo, si dicho entorno existe, el compilador no vuelve a agregar las funciones del .h. En el caso del ejemplo visto, el entorno se llama PERSONA_H y, en caso de hacer 2 veces #include "Persona.h", la definición de la clase sólo se incluiría una vez.

- **Definición de la clase**: Tal y como podemos ver, definimos la clase junto con todos sus métodos y datos miembro. También indicamos qué consideramos público y que consideramos privado.

Una vez definido el .h, este será el fichero que incluiremos en todos aquellos ficheros en donde vaya a utilizarse la clase. Para ello usaremos la directiva #include “nombreclase” que ya hemos utilizado para incluir algunas bibliotecas de uso frecuente.

A continuación, veremos cómo tenemos que definir el fichero .cpp asociado a la clase que estamos definiendo. En este caso, observaremos la implementación de algunas de las funciones de la clase Persona. Podemos ver dichas definiciones en la figura 8. En la figura 9 vemos el resto de funciones. Del fichero .cpp que implementa la clase debemos de tener en cuenta los siguientes puntos:

- El nombre del fichero .cpp debe ser el de la clase que a su vez debe de ser el mismo que el del fichero de la extensión .h. Es decir, debe llamarse Persona.cpp.

```
#ifndef PERSONA_H
#define PERSONA_H
#include <string>
using namespace std;
class Persona {
public:
    Persona(string newDNI, string newNombre, string newApellidos);
    Persona(const Persona& orig);
    string getDNI();
    string getNombre();
    string getApellidos();
    void setDNI(string newDNI);
    void setNombre(string neNombre);
    void setApellidos(string newApellidos);
    virtual ~Persona();
private:
    string DNI;
    string nombre;
    string apellidos;
};
#endif /* PERSONA_H */
```

Figura 7: Ejemplo de fichero .h

- Lo primero que debemos incluir en el fichero cpp es el .h asociado. Es posible que hayáis notado que a veces se utilizan los símbolos < y > y a veces, sin embargo, se utilizan las comillas. La diferencia principal es que con los símbolos < y >, el compilador busca el fichero .h en la librería del conjunto de librerías de C++ mientras que si ponemos comillas, el compilador busca la librería .h en los directorios del proyecto. Aunque usar siempre < y > para indicar la librería funciona siempre, es mejor utilizar las comillas para aquellos ficheros que creemos nosotros en el proyecto.

- Continuamos incluyendo todas las librerías que vayamos a utilizar en la implementación de las funciones o métodos. Es muy recomendable, para que todo funcione bien, añadir *using namespace std;* antes de comenzar la implementación de los métodos.

- A continuación, se lleva a cabo la implementación de las funciones o métodos uno a uno. Cuando trabajamos con una librería de funciones podemos definir las funciones tal y como ya conocemos. Sin embargo, si estamos implementando métodos de una clase debemos incluir el operador de ámbito que indica que a qué clase pertenece el método. Para ello, antes del nombre del método colocamos lo siguiente: <nombreClase>:: quedando la definición de la función como:

```
#include "Persona.h"
using namespace std;
Persona::Persona(string newDNI, string newNombre, string newApellidos) {
    DNI = newDNI;
    nombre = newNombre;
    apellidos = newApellidos;
}
string Persona::getDNI(){
    return DNI;
}
string Persona::getNombre(){
    return nombre;
}
string Persona::getApellidos(){
    return apellidos;
}
```

Figura 8: Ejemplo de fichero .cpp que implementa una clase.

<tipoDevuelto> <nombreClase>::<nombremétodo>(<parámetros>).

Finalmente, una vez que hemos podido definir los ficheros .h y .cpp sólo queda utilizarlos dentro del main. En la figura 10, podemos ver un ejemplo de un main.cpp que incluye la clase Persona junto con unas pruebas de dichas funciones.

Para el main.cpp debemos tener en cuenta lo siguiente:

- Debemos incluir todas las clases que vayamos a utilizar. Para ello utilizaremos la directiva #include y añadiremos el fichero .h de la clase o clases a utilizar.
- Una vez agregadas las clases, podemos utilizarlas de la misma manera en que lo haríamos si las hubiéramos definido en el mismo fichero.

4. Documentando un programa con Doxygen

4.1. Introducción

Tal y como hemos visto, la compilación separada permite la cómoda creación de programas de gran envergadura. Al distribuir el código entre varios ficheros, conseguimos dividir el propósito del programa en un conjunto de clases donde, cada una se encarga de llevar una de las tareas que sea necesario llevar a cabo. Esta división no sólo evita el tener que mantener largas líneas de código en un único fichero sino que ayuda en la paralelización del proceso de implementación del programa permitiendo que varios programadores

```
void Persona::setDNI(string newDNI){
    DNI = newDNI;
}
void Persona::setNombre(string newNombre){
    nombre = newNombre;
}
void Persona::setApellidos(string newApellidos){
    apellidos = newApellidos;
}
Persona::Persona(const Persona& orig) {
}
Persona::~Persona() {
}
```

Figura 9: Resto de funciones del fichero .cpp.

puedan trabajar en él a la vez, cada uno en un fichero. Esta metodología de programación fomenta, además, la modularización del programa que se está implementando.

Tal y como hemos comentado, la documentación de un programa ayuda a que varios programadores puedan trabajar a la vez de forma cómoda y entiendan el código realizado por sus compañeros. Sin embargo, para que el entendimiento sea posible, es necesario que todos los programadores que trabajan en el mismo código sigan una misma estructura en sus comentarios. De esta forma, les será mucho más sencillo comprender la funcionalidad del código que realizan y el trabajo que realicen será mucho más fluido. En esta sección veremos la herramienta Doxygen. Las directivas de doxygen permiten documentar un código de forma estructurada y estandarizada a la vez que permiten la generación de documentación del código comentado. Una de las grandes ventajas que tiene Doxygen es que no necesita ficheros aparte para incluir los comentarios, éstos se introducen en el mismo código comentado mejorando así también su lectura por parte de cualquier programador que quiera entender el código.

En la siguiente subsección estudiaremos cuáles son las directivas Doxygen y cómo podemos comentar los ficheros de nuestro programa utilizando esta herramienta.

4.2. Directivas

Como ya se vió, había dos maneras de introducir comentarios en un código:

- **Uso de //**: Servía para realizar comentarios en una línea.

```
#include <cstdlib>
#include <iostream>
#include "Persona.h"
using namespace std;

int main(int argc, char** argv) {
    Persona p("48568545G","Pedro", "Ramírez");
    p.setNombre("Lucas");
    p.setApellidos("Gómez");
    p.setDNI("1234567V");
    cout << p.getDNI() << ": " << p.getApellidos()
            << ", " << p.getNombre() << endl;
    return 0;
}
```

Figura 10: Ejemplo de fichero main.cpp.

- **Uso de /* y */**: Sirve para introducir un bloque de texto. Todo texto introducido y que esté entre ambos símbolos se considera comentado.

Para el uso de doxygen, sin embargo, utilizaremos los símbolos /** y */. Tal y como puede observarse, un código .cpp normal entendería dicha directiva como un comentario de bloque. Sin embargo, cuando usemos Doxygen, la herramienta entenderá esos símbolos como que ahí se encuentra un comentario Doxygen que debe de procesar. Dentro de los bloques de comentario Doxygen podemos introducir comentarios utilizando las siguientes directivas:

- **Directiva @file <nombreFichero>**: Al principio de un fichero debemos utilizar esta directiva para describir el propósito de dicho fichero.
- **Directiva @class <nombreClase>**: Con esta directiva indicamos que vamos a describir una clase.
- **Directiva @brief <descripción>**: Se utiliza para introducir una breve descripción del elemento comentado.
- **Directiva @param <parámetro> <descripción>**: Utilizamos esta directiva para introducir información detallada de un determinado parámetro.
- **Directiva @return <descripción>**: Realizamos una descripción general del valor devuelto por la función.
- **Directiva @retval <valor> <descripción>**: Indicamos, para cada posible valor devuelto, el motivo o razón de su obtención.

- **Directiva @pre <descripción>**: Se coloca antes de una función y sirve para describir su precondición.
- **Directiva @post <descripción>**: Se utiliza para indicar la postcondición de una determinada función.

Con el objetivo de que todos los conceptos vistos hasta ahora en el guión queden más claros, en la siguiente sección mostraremos en detalle un ejemplo completo de un código implementando mediante las técnicas vistas de compilación separada así como correctamente comentado utilizando las directivas de la herramienta Doxygen.

5. Ejemplo

En el ejemplo que pasaremos a detallar a continuación, implementaremos todo el código necesario para implementar un vector de pares que contenga varias funciones para su correcta manipulación. Consideraremos un par como un conjunto de dos enteros. Este proyecto está compuesto de dos clases diferentes. La primera es la clase Par mientras que la segunda utilizará la clase Par para implementar un vector de pares (objetos par). Los ficheros que necesitamos para llevar a cabo este proceso son los siguientes:

- **Par.h**: Contendrá la definición de la clase Par. Es decir, sus datos miembros y métodos.
- **Par.cpp**: Contendrá la implementación de los métodos de la clase Par.
- **VectorPares.h**: Contendrá la definición de la clase VectorPares.
- **VectorPares.cpp**: Contendrá la implementación de los métodos de la clase VectorPares.
- **main.cpp**: En este ejemplo, se introducirán ejemplos de uso de la clase VectorPares y Par.

Lo primero de todo es establecer las dependencias entre las diferentes clases. Estas dependencias serán importantes a la hora de establecer la sección de includes de los diferentes ficheros. En este caso, VectorPares necesita la clase Par para poder implementarse, ya que la usa para generar un vector de este tipo de objeto. Además, main.cpp, ya que utiliza tanto Par como VectorPares, necesitará la inclusión de ambos ficheros para su correcto funcionamiento. A continuación, pasaremos a ver cómo se implementan todos estos archivos, uno a uno. Nos centraremos tanto en su estructura como en la manera en que están comentados.

5.1. La clase Par

Veamos primero el fichero **Par.h**. La imagen 11 permite observar el código introducido antes de la definición de la clase. Tal y como podemos ver, no es necesario en este fichero incluir nada ya que sólo definimos la estructura de la clase. Al no usar, para ello, objetos externos, no es necesario incluir ningún fichero mediante la directiva include. Se pueden incluir en el .h los includes y el using namespace std;.

El comentario, utilizando la directiva class de doxygen, describe brevemente el objetivo y estructura de la clase. La directiva @file, se encarga de establecer el nombre del fichero que se está describiendo.

```
#ifndef PAR_H
#define PAR_H

/**
  @file Par.h
  @class Par
  Esta clase implementa el manejo de un par de enteros.
  Permite consultar y modificar cada entero por separado.
*/
class Par
{
```

Figura 11: Cabecera del fichero Par.h.

A continuación, nos centraremos en observar la estructura de la clase definida. Podemos verlo en la Figura 12.

Si analizamos la estructura, vemos como están incluidas todas las funciones relacionadas con la muestra y manipulación del objeto. Como ya se vio en FP, no es muy correcto incluir funciones tales como mostrar, que impriman por pantalla el objeto. Sin embargo, puede ser útil en fases intermedias de implementación, para facilitar la muestra de resultados y el testeo de los métodos implementados. Ese tipo de funciones luego deben borrarse en la entrega final. Por último, es importante comentar como la función suma se define fuera de la clase Par pero, sin embargo sí la ponemos dentro del fichero que define la clase. Un fichero .h puede definir clases, funciones, o conjuntos de clases y funciones. En este caso, al decidir implementar suma de pares como una función externa, es posible definirla dentro del fichero que define Par al ser una función íntimamente ligada con esta clase. Habría dos opciones más de implementación:

- Implementarla en un fichero util.h y util.cpp externo que contuviera todas las funciones externas a la clase pero relacionadas con el programa implementado.
- Implementar suma como parte de la clase.

Cada una de ellas tiene sus pros y sus contras no existiendo una única manera correcta de realizar esta implementación. Incluirla en un fichero externo util.h (u otro nombre) hace que los ficheros .h contengan exclusivamente las clases que se definen. Sin embargo, incluirla dentro del fichero que define la clase hace que el fichero .h contenga la clase y varias funciones externas relacionadas, lo cual es también una manera de programar organizada.

Pasemos ahora a ver el fichero **Par.cpp**. En este fichero implementaremos los métodos de la clase Par junto a todas las funciones externas a la clase asociadas. Recordemos que, si implementamos un método,

```
class Par
{
    public:
        Par();
        Par(int newx, int newy);
        ~Par();
        int verX();
        int verY();
        void setX(int newx);
        void setY(int newy);
        void mostrar();
    private:
    /**
    @brief Primer entero.
    */
    int x;
    /**
    @brief Segundo entero.
    */
    int y;
};
Par suma(Par uno, Par dos);

#endif // PAR_H
```

Figura 12: Estructura de la clase Par.h.

debemos utilizar el operador de ámbito :: para indicar que estamos implementando un método perteneciente a dicha clase. En caso contrario, no hará falta aplicarlo. Nótese que también es posible incluir bibliotecas en los ficheros .h pero cuando tiene más sentido hacerlo es cuando alguno de los objetos utilizados pertenece a otra clase. En la figura 13, podemos ver el inicio del fichero Par.cpp que incluye comentario global de la clase junto a la implementación de algunos constructores. Puede verse como es necesario incluir el fichero Par.h ya que se están implementando métodos definidos en ese fichero. Por último, observaremos la diferencia entre la implementación de la función externa suma con un método de la clase, mostrar, tal y como puede verse en la Figura 14. Si echamos un vistazo al código, podemos ver como la función suma, al no ser parte de la clase, se define sin el operador de ámbito Par::. Aunque esté fuera de la clase, es importante que su definición aparezca en el fichero .h para poder utilizarla cuando incluyamos la clase en otro cpp.

Con esto, terminamos por definir los ficheros relativos a la clase Par. Sin embargo, como ya hemos comentado, lo interesante de la compilación separada es la de poder incluir diferentes clases que, interactuando entre ellas, son capaces de crear códigos que sean a la vez organizados y complejos. A continuación, pasaremos a comentar la clase VectorPares que utilizará la clase Par a fin de implementar un vector de

```
#include "Par.h"
#include <iostream>

using namespace std;
/**
  @file Par.cpp
  @class Par
  Esta clase implementa el manejo para 2 valores de enteros, x e y.
*/

/**
@brief Constructor por defecto
@post Los valores miembro de la clase quedan inicializados a 0
*/
Par::Par()
{
    x=0;
    y=0;
}
```

Figura 13: Comentario inicial de la clase Par.cpp.

objetos tipo Par. Para ello, necesitaremos construir 2 ficheros: VectorPares.h y VectorPares.cpp.

5.2. La clase VectorPares

Comencemos analizando el fichero **VectorPares.h**. En este caso, es necesario incluir, tal y como se puede ver en la imagen 15, un include con la clase Par. Esto es porque uno de los datos miembro será un vector de pares. En cuanto a métodos incluiremos los operadores básicos de manejo de un vector: acceder a la posición, modificar valor, añadir al final y consultar tamaño actual. En su definición, usaremos memoria estática. Más adelante, veremos como realizar una definición más adecuada de esta clase mediante el uso de memoria dinámica. Tal y como se vio en FP, tendremos dos variables de tamaño. La primera, MAX, indicará el tamaño máximo reservado. Este tamaño no puede modificarse con lo que debe de ser un valor razonablemente alto. Sin embargo, es importante no exagerar su tamaño ya que estaremos reservando espacio inútil en memoria. Por otro lado, la variable tam indicará el tamaño actual del vector. Se deja a implementación del alumno los métodos elimina y eliminaUltimo, que mejorarían el manejo de la clase.

Por último, echemos un vistazo al fichero **VectorPares.cpp**, que incluye la implementación de los métodos de la clase VectorPares. Como incluimos el fichero VectorPares.h que contiene a su vez el fiche-

```
/**
@brief Muestra por pantalla los valores del Par. Utiliza el comando cout
*/
void Par::mostrar(){
    cout << x << " " << y << endl;
}

/**
@brief Suma los valores de dos pares. El x de un par de suma al x del segundo par y el y de un
par se suma con el y del otro par. Se genera un nuevo Par que almacena el resultado.
@param uno es uno de los pares que se suman.
@param dos es uno dea los pares que se suman.
@return resultado es el par que contiene la suma de ambos pares que entran como parámetro.
@pre los pares deben estar inicializados correctamente.
@post se genera un par resultado con valores válidos.
*/
Par suma(Par uno, Par dos){
    Par resultado;
    resultado.setX(uno.verX()+ dos.verX());
    resultado.setY(uno.verY()+dos.verY());
    return resultado;
}
```

Figura 14: Diferencia entre la implementación de un método de clase y una función externa en el fichero Par.cpp.

ro Par.h, no es necesario repetir la inclusión de este fichero. Sin embargo, dado que los ficheros .h están protegidos por la directiva ifndef, no pasaría nada si incluyéramos el fichero Par.h de nuevo. En la imagen 16, pueden verse dos funciones del fichero VectorPares.cpp comentadas utilizando la herramienta Doxygen.

5.3. Fichero main.cpp

Una vez implementadas las clases, creamos un fichero main.cpp en donde incluiremos el código inicial que ponga en marcha la interacción entre las clases. En este ejemplo, dicho código consistirá en una breve prueba que permitirá ver si los métodos de las clases funcionan correctamente. En un proyecto real, la función main de main.pp suele incluir las llamadas a métodos de clases que inician el proceso que se desea llevar a cabo. Cada clase debe estar centrada en una funcionalidad específica y, mediante la combinación de fichas funcionalidades, debe ser capaz de realizar el objetivo por el que hemos desarrollado el código. En la figura 17, podemos ver el código del fichero main.cpp cuyo único objetivo es llevar a cabo un testeo de los métodos de las clases implementadas. Tal y como puede verse, hemos usado la directiva include para incluir los ficheros .h de las 2 clases implementadas.

Los ficheros asociados al ejemplo completo comentado en esta sección están disponibles en Prado. Se recomienda que los bajéis, los abráis y los analicéis en detalle ya que contienen todas las directivas necesarias que necesitáis aprender para poder realizar proyectos informáticos con compilación separada de clases.

```
#ifndef VECTORPARES_H
#define VECTORPARES_H

#include "Par.h"

class VectorPares
{
    public:
        VectorPares();
        Par at(int i);
        void modifica(int i, Par newpar);
        void imprime();
        void anaide(Par newpar);
        int tamanioactual();

    private:
    const int MAX = 100;
    int tam = 0;
    Par pares[100];
};

#endif // VECTORPARES_H
```

Figura 15: Estructura de la clase VectorPares.h.

6. Entrega

La realización de esta práctica tiene un doble cometido:

- Creación de un proyecto de compilación separada a partir de un código que ha sido implementado en un único fichero cpp.
- Documentación del código mediante el uso de la herramienta Doxygen.

La entrega consistirá en un fichero .zip que contenga todos los ficheros del proyecto de programación. Dentro del zip que contiene la carpeta del proyecto, los ficheros deben estar organizados por carpetas según su tipo: los .h en un directorio include y los .cpp en un directorio src.

El código al que hay que aplicarse este proceso se encuentra colgado en la plataforma PRADO como codigounicofichero.cpp. El código incluido hace referencia al manejo de una baraja de cartas española. Se incluyen 2 clases y una función:

- **Clase Carta**: Representa una única carta. Cada carta está formada por un número (del 1 al 10) y un palo (oros, espadas, bastos o copas). El palo se representa con un valor de tipo enumerado. La definición del tipo enumerado puede hacerse dentro de Carta.h.

```
/**
@brief Añade un nuevo par al final del vector.
@param newpar Es el objeto par que añadimos al vector.
@pre Debe haber espacio reservado en el vector. Es decir, tam debe de ser menor que MAX-
@post el vector añade un nuevo par al final
*/
void VectorPares::anaide(Par newpar){
    if(tam < MAX){
        pares[tam]= newpar;
        tam++;
    }
}

/**
@brief Devuelve el tamaño actual del vector
@return Devuelve un entero que contiene el tamaño total del vector.
*/
int VectorPares::tamanioactual(){
    return tam;
}
```

Figura 16: Ejemplo de implementación de dos funciones de la clase VectorPares en VectorPares.cpp.

- **Clase Mano**: Representa un conjunto de cartas que es la mano del jugador. Una mano está compuesta por un número no fijo de cartas. Se incluyen funciones para eliminar una carta, añadir una carta a la mano y cambiar una carta por otra.

- **Función cartasComunes**: A partir de 2 manos, calcula el número de cartas que ambas manos tienen en común. Esta función se encuentra fuera de la clase Mano. A la hora de definir esta función puede hacerse en un fichero aparte o dentro del fichero Mano.cpp pero fuera de la clase. La cabecera de la función debe definirse en Mano.h y debe ponerse fuera también de la definición de la clase.

Se valorará que la separación sea correcta y siga los estándares indicados en el guión y que los comentarios realizados mediante la herramienta Doxygen sean precisos y útiles.

```
#include <iostream>
#include "Par.h"
#include "VectorPares.h"

using namespace std;

int main()
{
    //ejemplos para prueba
    Par prueba1(4,5);
    Par prueba2(6,6);
    Par prueba3;
    prueba3 = suma(prueba1,prueba2);
    prueba3.mostrar();

    //ejemplos para vectorpares
    VectorPares prueb;

    for(int i = 0; i < 10; i++){
        Par newpar(i,i+5);
        prueb.anaide(newpar);
    }
    Par modificapar(10,10);
    prueb.modifica(3,modificapar);
    prueb.imprime();
}
```

Figura 17: Código de la función main.cpp.